SC1001: Introduction to Computational Thinking

Week 12: Algorithm Design

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

EMNLP Best Paper

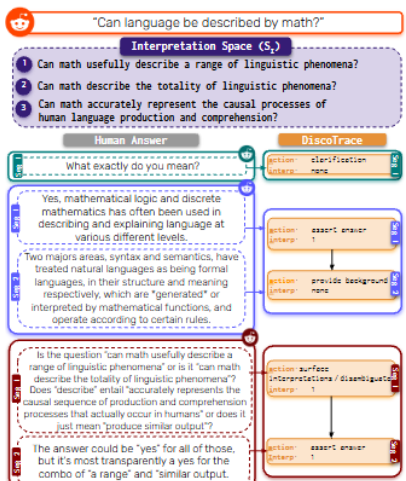
I missed the start of class because our paper won a Best Paper Award at EMNLP.



- The paper studies how language can be described by math.
- The method compares human answers with model reasoning traces.
- This is also algorithm design: define the representation, then reason about the steps.

EMNLP Best Paper

I missed the start of class because our paper won a Best Paper Award at EMNLP.



- The paper studies how language can be described by math.
- The method compares human answers with model reasoning traces.
- This is also algorithm design: define the representation, then reason about the steps.

Overview

1. Compare bubble sort and cocktail shaker sort
2. Use loop invariants to improve bounds
3. Separate useful examples from worst-case behavior
4. Add patience sorting as another quadratic worst case

Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

Step	Pass	List
Start	input	`[5, 9, 4, 6, 2, 7]`

Start with the unsorted list.

Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

Step	Pass	List
Start	input	`[5, 9, 4, 6, 2, 7]`
1	left to right	`[5, 4, 6, 2, 7, 9]`

The first left-to-right pass moves 9 to its final position; 2 moves left but still needs the backward pass.

Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

Step	Pass	List
Start	input	`[5, 9, 4, 6, 2, 7]`
1	left to right	`[5, 4, 6, 2, 7, 9]`
2	right to left	`[2, 5, 4, 6, 7, 9]`

The first right-to-left pass moves 2 to its final position.

Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

Step	Pass	List
Start	input	`[5, 9, 4, 6, 2, 7]`
1	left to right	`[5, 4, 6, 2, 7, 9]`
2	right to left	`[2, 5, 4, 6, 7, 9]`
3	left to right	`[2, 4, 5, 6, 7, 9]`

The next left-to-right pass moves 4 into place.

Cocktail Shaker Sort

Cocktail shaker sort alternates directions.

- A left-to-right pass moves large items toward the end.
- A right-to-left pass moves small items toward the beginning.
- The algorithm stops when a full pass makes no swaps.

Step	Pass	List
Start	input	`[5, 9, 4, 6, 2, 7]`
1	left to right	`[5, 4, 6, 2, 7, 9]`
2	right to left	`[2, 5, 4, 6, 7, 9]`
3	left to right	`[2, 4, 5, 6, 7, 9]`
4	no swaps	`[2, 4, 5, 6, 7, 9]`

The next pass makes no swaps, so the algorithm stops.

Problem 1: Python Code

Write Python code for cocktail shaker sort on an input list `values`.

Start from the bubble sort pattern, then add a right-to-left pass.

Cocktail Shaker Sort

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
```

Start with the function contract.

Cocktail Shaker Sort

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    while swapped:
        swapped = False
```

The outer loop keeps going while the last pass made a swap.

Cocktail Shaker Sort

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    while swapped:
        swapped = False
        for pos in range(0, len(values) - 1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
```

The forward pass is ordinary bubble sort.

Cocktail Shaker Sort

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    while swapped:
        swapped = False
        for pos in range(0, len(values) - 1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
        if not swapped:
            break
```

If the forward pass made no swaps, the list is already sorted.

Cocktail Shaker Sort

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    while swapped:
        swapped = False
        for pos in range(0, len(values) - 1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
        if not swapped:
            break
        swapped = False
        for pos in range(len(values) - 2, -1, -1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
```

The backward pass moves small values left.

Problem 2: Loop Invariants

State the two loop invariants for cocktail shaker sort.

- What is true after each left-to-right pass?
- What is true after each right-to-left pass?
- How can those facts improve the code?

Loop Invariants

- After the r-th left-to-right pass, the r largest items are in their final positions.

Loop Invariants

- After the r-th left-to-right pass, the r largest items are in their final positions.
- After the s-th right-to-left pass, the s smallest items are in their final positions.

Loop Invariants

- After the r-th left-to-right pass, the r largest items are in their final positions.
- After the s-th right-to-left pass, the s smallest items are in their final positions.
- Later passes can skip those settled regions.

Optimized Bounds

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    right_done = 0
    left_done = 0
```

Track how many values have settled on each side.

Optimized Bounds

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    right_done = 0
    left_done = 0
    while swapped:
        swapped = False
        for pos in range(left_done, len(values) - right_done - 1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
        right_done += 1
```

The forward pass skips the already-settled prefix and suffix.

Optimized Bounds

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    right_done = 0
    left_done = 0
    while swapped:
        swapped = False
        for pos in range(left_done, len(values) - right_done - 1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
        right_done += 1
        if not swapped:
            break
```

If nothing moved on the forward pass, stop.

Optimized Bounds

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    swapped = True
    right_done = 0
    left_done = 0
    while swapped:
        swapped = False
        for pos in range(left_done, len(values) - right_done - 1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
        right_done += 1
        if not swapped:
            break
        swapped = False
        for pos in range(len(values) - right_done - 1, left_done - 1, -1):
            if values[pos] > values[pos + 1]:
                values[pos], values[pos + 1] = values[pos + 1], values[pos]
                swapped = True
        left_done += 1
```

The backward pass uses the same bounds in the opposite direction.

Functional Decomposition

First separate one directional pass from the full algorithm.

```
from typing import Literal

Direction = Literal[-1, 1]

def cocktail_shaker_pass(
    values: list[int], start: int, stop: int, direction: Direction
) -> bool:
    """Moves values one step in the requested direction."""
    swapped = False

    if direction == 1:
        positions = range(start, stop - 1)
    else:
        positions = range(stop - 2, start - 1, -1)

    for pos in positions:
        if values[pos] > values[pos + 1]:
            values[pos], values[pos + 1] = values[pos + 1], values[pos]
            swapped = True

    return swapped
```

Helper-Based Version

The main loop now says what happens at the algorithm level.

```
def cocktail_shaker_sort(values: list[int]) -> None:
    """Sorts values in place."""
    left = 0
    right = len(values)
    swapped = True

    while swapped and right - left > 1:
        swapped = cocktail_shaker_pass(values, left, right, 1)
        right -= 1
        if not swapped:
            return

        swapped = cocktail_shaker_pass(values, left, right, -1)
        left += 1
```

Recursive Version

After both passes, recurse on the unsettled middle.

```
def cocktail_shaker_recursive(
    values: list[int], left: int = 0, right: int | None = None
) -> None:
    """Sorts values in place with recursive shrinking bounds."""
    if right is None:
        right = len(values)

    if right - left <= 1:
        return

    swapped = cocktail_shaker_pass(values, left, right, 1)
    right -= 1
    if not swapped:
        return

    swapped = cocktail_shaker_pass(values, left, right, -1)
    left += 1
    if not swapped:
        return

    cocktail_shaker_recursive(values, left, right)
```

Problem 3: Better Than Bubble Sort

Give an input list, or class of lists, where bubble sort is slow but cocktail shaker sort is asymptotically better.

Small Item at the End

Consider lists of the form

$$[k_2, k_3, \ldots, k_n, k_1] \quad \text{where} \quad k_1 < k_2 \leq \cdots \leq k_n.$$

Small Item at the End

Consider lists of the form

$$[k_2, k_3, \ldots, k_n, k_1] \quad \text{where} \quad k_1 < k_2 \leq \cdots \leq k_n.$$

- Cocktail shaker sort moves k_1 to the front during the first right-to-left pass.

Small Item at the End

Consider lists of the form

$$[k_2, k_3, \ldots, k_n, k_1] \quad \text{where} \quad k_1 < k_2 \leq \cdots \leq k_n.$$

- Cocktail shaker sort moves k_1 to the front during the first right-to-left pass.
- Bubble sort moves k_1 left by only one position per pass.

Small Item at the End

Consider lists of the form

$$[k_2, k_3, \ldots, k_n, k_1] \quad \text{where} \quad k_1 < k_2 \leq \cdots \leq k_n.$$

- Cocktail shaker sort moves k_1 to the front during the first right-to-left pass.
- Bubble sort moves k_1 left by only one position per pass.
- Cocktail shaker sort takes linear time on this class, while bubble sort takes quadratic time.

Counting Comparisons

Bubble sort

k_1 moves left one position per pass:

$$(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}.$$

n	Comparisons
16	120
64	2,016
1,024	523,776

Cocktail shaker sort

One forward pass, one backward pass, then one check pass:

$$(n-1)+(n-2)+(n-3).$$

n	Comparisons
16	42
64	186
1,024	3,066

This example class separates quadratic growth from linear growth.

Concrete Passes

Step	Comparison	List
Start	input	`[2, 2, 5, 9, 1]`

Start with the small item at the end.

Concrete Passes

Step	Comparison	List
Start	input	`[2, 2, 5, 9, 1]`
1	9 with 1	`[2, 2, 5, 1, 9]`

Forward pass: 9 moves right, but 1 only moves one step left.

Concrete Passes

Step	Comparison	List
Start	input	`[2, 2, 5, 9, 1]`
1	9 with 1	`[2, 2, 5, 1, 9]`
2	5 with 1	`[2, 2, 1, 5, 9]`

Backward pass step 1: compare 5 and 1.

Concrete Passes

Step	Comparison	List
Start	input	`[2, 2, 5, 9, 1]`
1	9 with 1	`[2, 2, 5, 1, 9]`
2	5 with 1	`[2, 2, 1, 5, 9]`
3	2 with 1	`[2, 1, 2, 5, 9]`

Backward pass step 2: compare the second 2 and 1.

Concrete Passes

Step	Comparison	List
Start	input	`[2, 2, 5, 9, 1]`
1	9 with 1	`[2, 2, 5, 1, 9]`
2	5 with 1	`[2, 2, 1, 5, 9]`
3	2 with 1	`[2, 1, 2, 5, 9]`
4	2 with 1	`[1, 2, 2, 5, 9]`

Backward pass step 3: compare the first 2 and 1. The small item reaches the front in one backward pass.

Problem 4: Worst Case

Argue that the worst-case time complexity of cocktail shaker sort is no better than $O(n^2)$.

Is this asymptotically better than bubble sort?

Worst-Case Class

Consider lists sorted in the wrong direction:

$$[k_n, k_{n-1}, \ldots, k_1] \quad \text{where} \quad k_1 < k_2 < \cdots < k_n.$$

Worst-Case Class

Consider lists sorted in the wrong direction:

$$[k_n, k_{n-1}, \ldots, k_1] \quad \text{where} \quad k_1 < k_2 < \cdots < k_n.$$

- The smallest item must move $n - 1$ positions.

Worst-Case Class

Consider lists sorted in the wrong direction:

$$[k_n, k_{n-1}, \ldots, k_1] \quad \text{where} \quad k_1 < k_2 < \cdots < k_n.$$

- The smallest item must move $n - 1$ positions.
- The next smallest item must move $n - 2$ positions.

Worst-Case Class

Consider lists sorted in the wrong direction:

$$[k_n, k_{n-1}, \ldots, k_1] \quad \text{where} \quad k_1 < k_2 < \cdots < k_n.$$

- The smallest item must move $n-1$ positions.
- The next smallest item must move $n-2$ positions.
- The total number of required adjacent swaps is

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

Worst-Case Class

Consider lists sorted in the wrong direction:

$$[k_n, k_{n-1}, \ldots, k_1] \quad \text{where} \quad k_1 < k_2 < \cdots < k_n.$$

- The smallest item must move $n-1$ positions.
- The next smallest item must move $n-2$ positions.
- The total number of required adjacent swaps is

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

- The worst case is therefore quadratic, just like bubble sort.

Problem 5: Patience Sorting

Patience sorting uses piles, similar to a card game.

In the US, this game is usually called Solitaire.

For each item:

1. Put it on the leftmost pile whose top item is at least as large.
2. If no such pile exists, start a new pile.

After building the piles, repeatedly remove the smallest visible pile top to produce the sorted output.



Image: Maaaks, GPL, Wikimedia Commons, curid 6952483.

Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$



Place 5 on the first pile.

Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

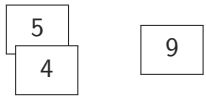


9 is larger than the first pile top, so it starts a new pile.

Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

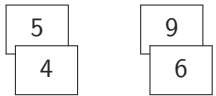


4 goes on the leftmost pile whose top is at least 4.

Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

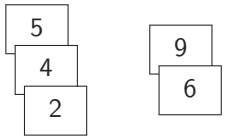


6 goes on 9, becoming the new visible top.

Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$

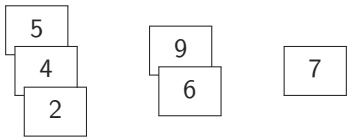


2 goes on the first pile, becoming the new visible top.

Patience Sorting Demo

Use the same six-item list:

$$[5, 9, 4, 6, 2, 7].$$



7 starts a third pile.

Problem 5: Discussion

Write Python code for the simple version of patience sorting.

- Scan pile tops from left to right to place each item.
- Scan pile tops to find the smallest visible item during output.
- Give a worst-case input class for this simple implementation.

Patience Sorting: Build Piles

```
def patience_sort(values: list[int]) -> list[int]:
    """Returns values in sorted order."""
    piles = []
```

Start with an empty list of piles.

Patience Sorting: Build Piles

```
def patience_sort(values: list[int]) -> list[int]:
    """Returns values in sorted order."""
    piles = []
    for value in values:
        placed = False
```

Process one input value at a time.

Patience Sorting: Build Piles

```
def patience_sort(values: list[int]) -> list[int]:
    """Returns values in sorted order."""
    piles = []
    for value in values:
        placed = False
        for pile in piles:
            if pile[-1] >= value:
                pile.append(value)
                placed = True
                break
```

Scan pile tops until one can accept the value.

Patience Sorting: Build Piles

```
def patience_sort(values: list[int]) -> list[int]:
    """Returns values in sorted order."""
    piles = []
    for value in values:
        placed = False
        for pile in piles:
            if pile[-1] >= value:
                pile.append(value)
                placed = True
                break
        if not placed:
            piles.append([value])
```

If no pile accepts the value, start a new pile.

Smallest Visible Pile Top

```
def index_of_pile_with_smallest_top(piles: list[list[int]]) -> int:
    """Returns the index of the pile with the smallest top."""
    best = 0
```

Start by assuming the first pile has the smallest visible top.

Smallest Visible Pile Top

```
def index_of_pile_with_smallest_top(piles: list[list[int]]) -> int:
    """Returns the index of the pile with the smallest top."""
    best = 0
    for pos in range(1, len(piles)):
        if piles[pos][-1] < piles[best][-1]:
            best = pos
```

Scan the rest of the pile tops and update the best index when needed.

Smallest Visible Pile Top

```
def index_of_pile_with_smallest_top(piles: list[list[int]]) -> int:
    """Returns the index of the pile with the smallest top."""
    best = 0
    for pos in range(1, len(piles)):
        if piles[pos][-1] < piles[best][-1]:
            best = pos
    return best
```

Return the position of the pile to pop from.

Patience Sorting: Finish Function

```
def patience_sort(values: list[int]) -> list[int]:
    """Returns values in sorted order."""
    piles = []
    for value in values:
        placed = False
        for pile in piles:
            if pile[-1] >= value:
                pile.append(value)
                placed = True
                break
        if not placed:
            piles.append([value])

    output = []
    while piles:
```

Keep removing visible pile tops until all piles are empty.

Patience Sorting: Finish Function

```
def patience_sort(values: list[int]) -> list[int]:
    """Returns values in sorted order."""
    piles = []
    for value in values:
        placed = False
        for pile in piles:
            if pile[-1] >= value:
                pile.append(value)
                placed = True
                break
        if not placed:
            piles.append([value])

    output = []
    while piles:
        best = index_of_pile_with_smallest_top(piles)
        output.append(piles[best].pop())
```

Each output step scans pile tops and removes the smallest visible value.

Patience Sorting: Finish Function

```
def patience_sort(values: list[int]) -> list[int]:
    """Returns values in sorted order."""
    piles = []
    for value in values:
        placed = False
        for pile in piles:
            if pile[-1] >= value:
                pile.append(value)
                placed = True
                break
        if not placed:
            piles.append([value])

    output = []
    while piles:
        best = index_of_pile_with_smallest_top(piles)
        output.append(piles[best].pop())
        if piles[best] == []:
            del piles[best]
    return output
```

Remove empty piles, then return the sorted output.

Patience Sorting: Worst Case

Consider an already sorted list:

$$[1, 2, 3, \ldots, n].$$

Patience Sorting: Worst Case

Consider an already sorted list:

$$[1, 2, 3, \ldots, n].$$

- Each new item starts a new pile.

Patience Sorting: Worst Case

Consider an already sorted list:

$$[1, 2, 3, \ldots, n].$$

- Each new item starts a new pile.
- The build phase scans many pile tops before creating the next pile.

Patience Sorting: Worst Case

Consider an already sorted list:

$$[1, 2, 3, \ldots, n].$$

- Each new item starts a new pile.
- The build phase scans many pile tops before creating the next pile.
- The simple output phase also scans many pile tops.

Patience Sorting: Worst Case

Consider an already sorted list:

$$[1, 2, 3, \ldots, n].$$

- Each new item starts a new pile.
- The build phase scans many pile tops before creating the next pile.
- The simple output phase also scans many pile tops.
- This simple implementation has quadratic worst-case time.

Patience Sorting: Worst Case

Consider an already sorted list:

$$[1, 2, 3, \ldots, n].$$

- Each new item starts a new pile.
- The build phase scans many pile tops before creating the next pile.
- The simple output phase also scans many pile tops.
- This simple implementation has quadratic worst-case time.
- With better pile placement and a priority queue for the pile tops, patience sort can be $O(n \log n)$ instead of repeated scans.

Takeaways

- Bidirectional passes can improve some inputs.
- Loop invariants justify smaller loop bounds.
- A better example class does not imply a better worst case.
- Cocktail shaker sort and simple patience sorting both have quadratic worst cases.

Takeaways

- Bidirectional passes can improve some inputs.
- Loop invariants justify smaller loop bounds.
- A better example class does not imply a better worst case.
- Cocktail shaker sort and simple patience sorting both have quadratic worst cases.
- Patience Sort is how I sort physical items (exams / Pokémon cards)